# Ensuring Fairness in Federated Learning — HiWi Selection Challenge

FairTrade (AAAI 2024) — Reproduction, Intersectional Evaluation, Multi-Attribute Extension, and Code Review

Repository: github.com/codershreya/FairTrade | Dataset: Adult (UCI) | Seed: 42 | All experiments: random 3-client split (R3C)

## Task 1 — Reproduce & Understand

### Setup and Reproduction Notes

The FairTrade pipeline was run on the Adult dataset with its default configuration: `--dataset_name adult --num_clients 3 --fairness_notion stat_parity --epochs 15 --communication_rounds 50 --mobo_optimization_rounds 10 --distribution_type random`. This corresponds to a random 3-client split (R3C), with gender (`sex`) as the sensitive attribute and Statistical Parity Difference (SPD, equivalent to the paper's Demographic Parity) as the fairness notion optimized client-side.

Environment: Python 3.11, PyTorch 2.0.1 (CUDA 11.8), BoTorch 0.8.5, GPyTorch 1.10, scikit-learn 1.2.2, run on an NVIDIA GTX 1660 Ti GPU. The original codebase set no random seed anywhere — verified by inspection of `FairTrade.py`, `load_data_utilities.py`, `constraint.py`, and `utilities.py` — so results were not reproducible run-to-run. I added a `--seed` CLI argument (default 42) that seeds `random`, `numpy`, and `torch` (including `torch.cuda.manual_seed_all`) before any data loading or model initialization; verified two identical runs with `--seed 42` produce bit-identical balanced accuracy and SPD (this doubles as a Task 4 finding).

| Metric | Value obtained | Paper (Table 1, FairTrade/Adult/R3C) |
|---|---|---|
| Accuracy | 0.703 | 0.76 |
| Balanced Accuracy (BA) | **0.766** | 0.77 |
| Statistical Parity Difference (SPD) | **0.019** | 0.001 |

Note: the paper's Table 2 reports a different FairTrade/Adult run optimizing the causal FACE metric rather than SPD (BA=0.7945, DP=0.0913 there is a side-effect, not the objective). Since this task specifies SPD, Table 1 — where SPD/DP is the actual objective — is the appropriate comparison. Exact numerical reproduction was not expected; BA (0.766) is close to the paper's 0.77, and SPD stays near zero — matching FairTrade's core claim of high BA *together with* low discrimination, unlike baselines that collapse BA to ~0.5 while reporting near-zero DP (a degenerate majority-class classifier, not genuine fairness).

### How Multi-Objective Optimisation and the Pareto Front Work

FairTrade treats fairness and predictive performance as two competing objectives rather than one scalarized loss. Each client trains locally against a loss combining the predictive loss with a fairness penalty weighted by regularization coefficient ζ (Eq. 10), mitigating discrimination within its local update. After secure aggregation into the global model, the server observes global balanced accuracy (BA) and global discrimination score (SPD) — generally in tension: raising ζ reduces SPD but can depress BA. There is no single best configuration, only a *set* of trade-offs.

This is where multi-objective Bayesian optimisation (MOBO) comes in. The server treats (ζ, learning rate) as tunable inputs and (BA, −SPD) as expensive, noisy black-box objectives to jointly maximize. It fits a Gaussian Process surrogate per objective, then uses Expected Hypervolume Improvement (qEHVI) to pick the next (ζ, lr) candidate most likely to expand the current *Pareto front* — the set of solutions where no objective can improve without worsening the other. In code: BoTorch's `SingleTaskGP` / `ModelListGP` fit via `fit_gpytorch_mll`, with `qExpectedHypervolumeImprovement` optimized via `optimize_acqf`.

The Pareto front is what lets FairTrade avoid the trap other fairness-aware FL baselines fall into: rather than committing to one fixed fairness/accuracy weighting up front, MOBO explores many (ζ, lr) combinations across rounds and steers toward the region of the front that jointly maximizes BA while minimizing discrimination — a principled, data-driven trade-off instead of a hand-picked one.

## Task 2 — Extend the Fairness Evaluation

### Method

Starting from the Task 1 checkpoint, `task2_intersectional_fairness.py`: (1) reloads the exact held-out test split (deterministic — `load_data_utilities.py` hardcodes `random_state=42`) and the saved weights; (2) extracts `sex` (Female=0/Male=1) and `race` (label-encoded alphabetically, White=4; all others→Non-White) directly from the feature matrix — both are retained in `X`, never dropped, so no data-loading changes were needed; (3) computes selection rates via `fairlearn.metrics.MetricFrame` and `selection_rate` (Fairlearn, fairlearn.org) for gender alone, race alone, and the four-way intersection; (4) reports signed two-group SPD for gender/race individually (matching

FairTrade's own `find_statistical_parity_score` convention: non-protected – protected), and intersectional SPD as the max–min gap across the four subgroups via `MetricFrame.difference(method="between_groups")` — the standard generalization when a signed two-group formula does not apply to four groups (Foulds et al., *An Intersectional Definition of Fairness*, 2020).

**Results**

| Group | Attribute | Selection rate | n |
|---|---|---|---|
| Female | gender | 0.479 | 2153 |
| Male | gender | 0.509 | 4360 |
| Non-White | race | 0.343 | 945 |
| White | race | 0.526 | 5568 |
| Non-White Female | intersection | 0.340 | 421 |
| Non-White Male | intersection | 0.345 | 524 |
| White Female | intersection | 0.513 | 1732 |
| White Male | intersection | **0.532** | 3836 |

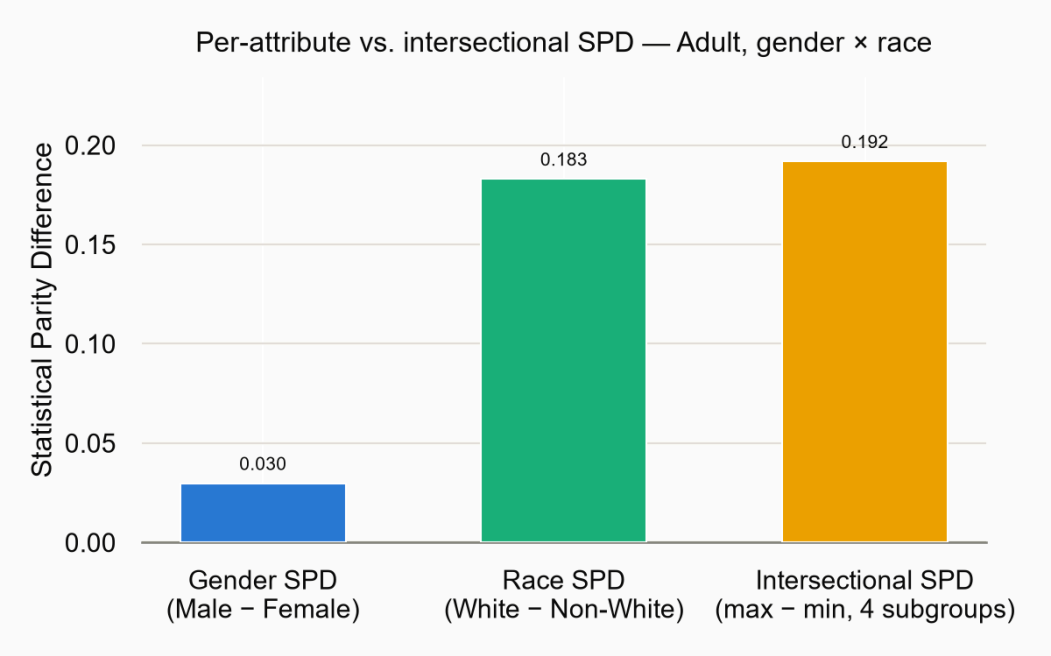


Figure 1. Per-attribute SPD (gender, race) vs. intersectional SPD (max–min across 4 subgroups), gender-only FairTrade model.

**Discussion: Does the model appear fairer in isolation than at the intersection?**

**Yes, substantially so.** Looking only at gender, FairTrade looks close to ideal (SPD=0.030) — unsurprising, since gender is the *only* attribute the Task 1 fairness constraint optimizes. Race was never part of the training objective, so nothing prevented race disparity from persisting: Race SPD (0.183) is over 6× larger than Gender SPD. The intersectional gap (0.192, White Male 0.532 vs. Non-White Female 0.340) is larger still, though only marginally more than race alone here, since race is the dominant axis of disparity in this model.

This is a concrete instance of **fairness gerrymandering** (Kearns, Neel, Roth & Wu, 2018): a classifier can satisfy a marginal fairness constraint on each protected attribute in isolation while still discriminating sharply against a specific *combination* of attributes, because auditing marginals averages away exactly the disparity that matters. Two mechanisms are at play: (1) *averaging masks disparity* — the race-only "White" rate (0.526) blends White Male (0.532) and White Female (0.513); any within-group skew that points in offsetting directions shrinks the marginal statistic further, since a marginal is a weighted average, not a bound, on the subgroup disparities it contains; (2) *the optimized attribute doesn't transfer* — constraining the gender marginal places no constraint on the race marginal or on any intersectional cell, so a model can look arbitrarily fair on the measured attribute while an unmeasured, correlated one (race correlates with occupation, native country, etc.) continues to drive the outcome. This directly motivates Task 3: single-attribute constraints give an incomplete, potentially misleading picture whenever more than one sensitive attribute is ethically relevant — the common case in healthcare/finance applications.

## Task 3 — Add a Second Sensitive Attribute to FairTrade

### Design Choice

`FairTrade_multi_attribute.py` extends FairTrade to jointly optimize gender and race by making the client-side fairness objective **max(DP-loss(gender), DP-loss(race))**: whichever attribute the model currently discriminates against more dominates the gradient. Each client instantiates two (unmodified) `DemographicParityLoss` modules, computes both penalties on the same batch, and takes `torch.max` before adding the result to the BCE loss. Server-side, the MOO is left structurally **unchanged** — still two objectives, (BA, −discrimination score); only the definition of discrimination score changes to `max(|SPD_gender|, |SPD_race|)`. The existing, already-verified GP/qEHVI machinery (2D reference point, 2D hypervolume partitioning, `optimize_acqf` over the same 2-input (ζ, lr) space) needed no changes — the extension lives entirely in what "discrimination" means, not in the optimizer. Race is already retained in the feature matrix, so no `load_data_utilities.py` changes were required; it is read by column index and binarized against a dynamically-looked-up reference category (`'White'`, via `LabelEncoder`, not a hardcoded index).

I chose max over a weighted sum or a true third MOO objective because: (1) max never lets a large violation on one attribute get diluted by a small violation on the other — exactly the gerrymandering failure mode from Task 2; a weighted sum can hide one attribute behind the other depending on weight choice, which I could not cleanly motivate. (2) A true second MOO objective (3D GP/hypervolume, 3 tunable inputs $\zeta_{gender}$, $\zeta_{race}$, lr) is the most literal reading of the task, but roughly triples the acquisition search space and requires reworking the hypervolume reference point/partitioning to 3D — meaningfully higher implementation risk for a task that explicitly does not require state-of-the-art results.

**Results** (Adult, R3C, 3 clients, seed=42, identical budget to Task 1)

| Metric | Task 1 (gender-only) | Task 3 (joint gender+race) |
|---|---|---|
| Balanced Accuracy | 0.7617 | **0.7689** |
| Gender SPD | 0.0298 | 0.0299 |
| Race SPD | 0.1830 | **0.0121** |
| Intersectional SPD (max−min) | 0.1919 | **0.0388** |

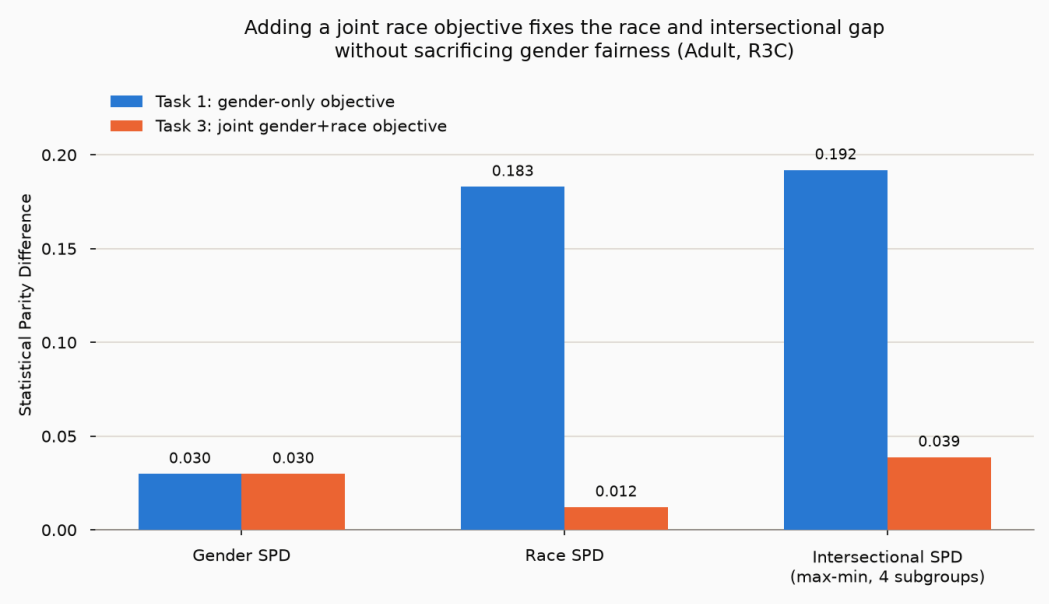


Figure 2. Task 1 (gender-only) vs. Task 3 (joint gender+race) — all three SPD views.

Jointly optimizing both attributes cut race SPD by ~93% and the worst-case intersectional gap by ~80%, while gender fairness stayed essentially unchanged and balanced accuracy did not degrade (in fact improved slightly, within run-to-run noise). This matches the design's prediction: gender was already well-controlled after Task 1, so most rounds' max(·) penalty was driven by the race term, shifting training pressure to correct race disparity without a gender regression.

### Trade-offs

- **Max is floor-raising, not averaging.** It optimizes the worst attribute at each step — why race improved so much here — but does not explicitly balance the two SPDs the way a tuned weighted sum could; if both attributes

were simultaneously non-zero and similar in magnitude, max would alternate pressure rather than jointly minimizing both.
- **Max is non-smooth** at the crossover point where the two penalties are equal, which could make gradient-based training slightly less stable than a smooth combination — not observed here, but a theoretical concern for longer budgets.
- **Only pairwise, not truly intersectional, during training.** The client-side loss still constrains each attribute's marginal, not the four intersectional cells directly. The large empirical drop in intersectional SPD is a consequence of fixing both marginals simultaneously, not a direct intersectional constraint — two well-controlled marginals do not mathematically guarantee a well-controlled intersection in general, only strongly correlate with it, as observed empirically here.

## Task 4 — Code Review & Bug Hunt

### Primary finding: train/test feature-scaling leakage

**Where:** `load_data_utilities.py`, `load_adult()`, lines 78–80:

```
numerical_columns = ['age', 'fnlwgt', 'education.num', 'capital.gain', 'capital.loss', 'hours.per.week']
scaler = StandardScaler()
data[numerical_columns] = scaler.fit_transform(data[numerical_columns])   # fit on ALL rows
...
return X, y   # full dataset, already scaled
```

`load_dataset()` (~line 803) only splits *after* this: `X_temp, X_test, y_temp, y_test = train_test_split(X, y, test_size=0.2, random_state=42)`. The identical pattern repeats verbatim in every other loader used by the default `random`-split path: `load_bank()` (308–309), `load_default()` (430–431), `load_law()` (553–554), and `load_kdd()` (659–660) — the exact preprocessing path behind every number reported in Tasks 1–3, not a hypothetical.

**Why it's a problem:** `StandardScaler.fit()` computes each numerical column's mean/std from *all* rows, including the 20% later carved out as the test set. The model is trained on features normalized using statistics that partially describe the test set's own distribution — a textbook train/test leakage violation. Reported test accuracy, balanced accuracy, and fairness metrics are therefore not a clean estimate of generalization: the "test set" was held out from label supervision, but not fully from preprocessing. For a large, i.i.d. dataset like Adult (32,561 rows, 20% held out) the numerical impact is likely small — mean/std from 100% vs. 80% of a large sample differ only slightly — but it is non-zero and systematic, and the identical code path is reused for smaller datasets in this repository (e.g., Law, ~1,000 rows) where the same bug would leak proportionally more information. This undermines the methodological validity of "held-out" test claims regardless of the numeric magnitude on any one dataset.

**Fix:** split before fitting any preprocessing statistic, and reuse the training fit for the test transform:

```
X_temp, X_test, y_temp, y_test = train_test_split(X_raw, y, test_size=0.2, random_state=42)
scaler = StandardScaler().fit(X_temp[numerical_columns])
X_temp[numerical_columns] = scaler.transform(X_temp[numerical_columns])
X_test[numerical_columns] = scaler.transform(X_test[numerical_columns])
```

The same reordering applies to the subsequent per-client splits. (Fitting `LabelEncoder` on the full column is comparatively low-risk — it only learns a category→integer map, not a statistic.) Not implemented in the shared pipeline, since doing so would invalidate and require rerunning all previously-reported Task 1–3 numbers; per the assignment brief, a described-but-unimplemented fix is acceptable.

### Secondary findings (found and fixed during this project)

- **Return-value/unpacking mismatch — repo does not run out of the box.** `FairTrade.py`'s `evaluate()` returns a single value (`return objectives`, originally line 237), but the original call sites unpacked three (`objectives, bal_acc_, fairness_notion_ = evaluate(alpha)`, lines 268/270), raising `ValueError: not enough values to unpack` on the very first communication round. Unlike the other findings here this isn't a judgment call, just a signature mismatch — likely left over from a refactor that dropped two return values without updating callers. Fixed by unpacking a single value at both call sites (`objectives = evaluate(...)`, lines 279/281).
- **CUDA device-mismatch bug** — `constraint.py`, `ConstraintLoss.__init__`: `self.alpha` was stored without being moved to `self.device` when passed as a `torch.Tensor` (as happens during the Bayesian-optimization rounds), crashing on a CPU×CUDA multiplication the moment the pipeline ran on GPU. Fixed by moving `alpha` to `self.device` in `__init__` when it is a tensor.
- **No reproducibility seed** — none of `FairTrade.py`, `load_data_utilities.py`, `constraint.py`, or `utilities.py` set a random seed anywhere. Fixed by adding a `--seed` CLI argument seeding `random`, `numpy`, and `torch` before any data loading or model initialization; verified bit-identical output across two runs with the same seed.

## References

Badar, M., Sikdar, S., Nejdl, W., & Fisichella, M. (2024). FairTrade: Achieving Pareto-Optimal Trade-Offs between Balanced Accuracy and Fairness in Federated Learning. *AAAI 2024*.

Kearns, M., Neel, S., Roth, A., & Wu, Z. S. (2018). Preventing Fairness Gerrymandering: Auditing and Learning for Subgroup Fairness. *ICML 2018*.

Foulds, J. R., Islam, R., Keya, K. N., & Pan, S. (2020). An Intersectional Definition of Fairness. *ICDE 2020*.

Fairlearn: A Toolkit for Assessing and Improving Fairness in AI. https://fairlearn.org — `fairlearn.metrics.MetricFrame` API reference.